FIAP ON

COLHEITA DE SOLUÇÕES INTELIGENTES - TRANSFORMANDO DADOS EM AÇÕES

DESMISTIFICANDO O SERVERLESS: SOLUÇÕES INOVADORAS NA AWS

12

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Projeto de uma Solução Serverless 7
Figura 2 - Acesso à função Lambda 9
Figura 3 - Criando uma nova função Lambda 10
Figura 4 - Informações da função Lambda 11
Figura 5 - Função Lambda criada 12
Figura 6 - Deploy e Teste de uma função Lambda 13
Figura 7 - Teste da Função Lambda 14
Figura 8 - Arquitetura da solução, em destaque laranja, tem-se a ingestão de Dados 15
Figura 9 - Detalhamento da Solução 15
Figura 10 - Buckets criados 17
Figura 11 - Criar gatilho 22
Figura 12 - Gatilho para o bucket S3 23
Figura 13 - Acesso ao bucket de entrada de dados 24
Figura 14 - Carga do arquivo CSV no bucket 25
Figura 15 - Arquivo processado 25
Figura 16 - Conteúdo do arquivo JSON gerado 26
Figura 17 - Ingestão de Dados 29
Figura 18 - Consumo de dados por meio de API, no destaque em laranja 30
Figura 19 - Troca de informações por API 30
Figura 20 - Detalhamento da solução de consumo 33
Figura 21 - Criação da API 37
Figura 22 - Criar API HTTP 37
Figura 23 - Criação da API HTTP 38
Figura 24 - Criação de Rotas 38
Figura 25 - Escolha do método HTTP e caminho da rota 39
Figura 26 - Rotas criadas 40
Figura 27 - Anexar Integração 40
Figura 28 - Detalhes da integração 41
Figura 29 - Anexar integração existente 41
Figura 30 - Rotas com integrações com função Lambda 42
Figura 31 - *Endpoint* da API 42
Figura 32 - Simulação de consulta com o navegador 43
Figura 33 - Consulta de registros por data pela API 43
Figura 34 - Consulta por data pelo Postman 44
Figura 35 - Exclusão da API 44
Figura 36 - Exclusão da tabela do DynamoDB 45

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE TABELAS

Tabela 1 - Conteúdo dos arquivos de teste.....24

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Função Lambda básica 12
Código-fonte 2 - Exemplo de resposta se a função foi acionada 13
Código-fonte 3 - Função Lambda que processa arquivos CSV.....19
Código-fonte 4 - Exemplo de arquivo CSV sendo carregado na entrada para ser convertido para JSON 21
Código-fonte 5 - Exemplo de conteúdo do CSV.....21
Código-fonte 6 - Exemplo de saída após a conversão para JSON 22
Código-fonte 7 - Função Lambda para gravar dados no DynamoDB.....28
Código-fonte 8 - Função Lambda para consumo da API 36

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 DESMISTIFICANDO O SERVERLESS: SOLUÇÕES INOVADORAS NA AWS 6
1.1 Arquitetura Serverless 6
1.2 Projeto real 7
2 INGESTÃO E TRANSFORMAÇÃO DE DADOS NA NUVEM 8
2.1 AWS Lambda 8
2.2 Testes da função Lambda 12
2.3 Transformação de dados com funções Lambda 14
2.4 Disparo automático de funções Lambda (*triggers*) 16
2.5 Ingestão de dados com funções Lambda 26
3 CONSUMO DE DADOS NA NUVEM 30
3.1 API Gateway 31
3.2 Métodos HTTP 32
3.3 Consumo da API 33
3.4 Limpeza dos recursos utilizados 44
CONCLUSÃO 46
REFERÊNCIAS 47
GLOSSÁRIO 48

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 DESMISTIFICANDO O SERVERLESS: SOLUÇÕES INOVADORAS NA AWS

1.1 Arquitetura Serverless

Uma arquitetura *serverless* é um modelo de computação em nuvem na qual podemos disponibilizar aplicações completas sem nos preocuparmos em gerenciar e configurar servidores.

A infraestrutura e a escalabilidade são totalmente gerenciadas pelo provedor de serviços de nuvem, como a AWS, permitindo que os desenvolvedores foquem no problema de negócio e no código que precisam desenvolver.

Além de não se preocupar com a gestão de servidores, a forma de cobrança é simplificada, não havendo cobrança por servidores ociosos. O pagamento é feito sob demanda de acordo com o que é solicitado.

A arquitetura *serverless* permite a experimentação e testes para projetos de pequeno porte, incluindo a disponibilização de websites, o processamento assíncrono ou o processamento de dados de streaming ou Internet das Coisas (IOT — Internet of Things).

Algumas vantagens são:

- **Redução de custos:** não há custo com ociosidade de computadores.
- **Alta disponibilidade e resiliência:** a infraestrutura é projetada para tolerar falhas e garantir alta disponibilidade.
- **Foco no negócio:** o desenvolvedor por focar em resolver os problemas de negócio ao invés de focar na gestão de infraestrutura.
- **Escalabilidade:** as soluções ganham escalabilidade, sendo capazes de atender a um número variável de usuários e demandas.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.2 Projeto real

A partir desse ponto, vamos aplicar uma solução real para um sistema Serverless. Esse exemplo é totalmente aplicável em sua vida profissional como arquiteto(a) de nuvem. Imagine que você receberá dados de sensores que medem a temperatura de hora em hora. Os dados chegarão em um bucket S3 e deverão ser alimentados automaticamente em um banco de dados. Em seguida, serão disponibilizados para consulta para os nossos clientes.

A seguir, podemos visualizar o diagrama do projeto de uma solução *serverless* que atenda às nossas necessidades de negócio.

Figura 1 - Projeto de uma Solução Serverless
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

No passo 1 da solução, vamos simular a ingestão de dados e receber um arquivo CSV de um fornecedor. O conteúdo do arquivo deverá ser processado e armazenado em um bucket S3. Ao receber os dados, uma função Lambda irá ser acionada automaticamente para fazer a carga em um banco de dados, como o DynamoDB.

No passo 2 da solução, iremos preparar o consumo de dados, disponibilizando uma API de consulta e utilizando o serviço API Gateway, que permite criar APIs REST. Neste serviço, disponibilizaremos um endereço de consulta dos dados por dia aos nossos clientes.

Para implantar esta solução, iremos abordar as características dos serviços envolvidos, e como podemos acionar a automação destes serviços.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

2 INGESTÃO E TRANSFORMAÇÃO DE DADOS NA NUVEM

Considerando a primeira parte de nossa solução completa de ingestão de dados, precisamos de um serviço da AWS que consiga processar um arquivo de entrada, filtrando os dados necessários, para depois gravar no banco de dados. Esta opção é o AWS Lambda. Inicialmente veremos alguns conceitos básicos e, em seguida, iremos aplicá-los ao nosso cenário.

2.1 AWS Lambda

O serviço AWS Lambda permite executar trechos de código em resposta a eventos (*event-driven*), sem se preocupar em gerenciar e configurar servidores (*serverless*).

É possível executar códigos sob demanda em diferentes linguagens de programação, como Python, nodejs, Java, go, .net, entre outros.

A cobrança é feita somente a partir de cada execução de código. Portanto, paga-se de acordo com a quantidade de chamadas realizadas, considerando o tempo de execução do código e a quantidade de memória e CPU envolvidas.

O desenvolvedor pode escrever seu código diretamente no console da AWS ou realizar o upload de um arquivo ZIP, e o Lambda automaticamente aloca os recursos necessários para executar o código.

O Lambda pode ser acionado por meio de eventos de outros serviços, como a carga de arquivos em um bucket S3. Este acionamento é chamado de **gatilho** (*triggers*).

No entanto, o Lambda tem algumas restrições como:

- **Alocação de memória:** 128MB a 10 GB;
- **Execuções simultâneas:** 1000;
- **Tempo máximo de execução:** 15 minutos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Para soluções que exigem mais recursos, como por exemplo, se o seu programa Python leva mais que 15 minutos para filtrar a entrada, o Lambda não é adequado. Da mesma forma, se o arquivo que você carregou, como um vídeo, possui mais de 10 GB para ser processado, o Lambda também não irá suportar. Ou, se o seu programa precisa manter estados entre execuções, isto é, transferir valor de variáveis internas ou continuar o processamento em outro programa Python alocado em um segundo Lambda, o primeiro Lambda nem o segundo conseguirão suportar, pois cada invocação ocorre em um ambiente efêmero.

Um ambiente efêmero é um ambiente de execução que não mantém estado entre execuções e é descartado após o uso. Isso significa que, toda vez que uma função é invocada, ela pode ser executada em um novo ambiente, sem acessar arquivos, variáveis ou configurações das execuções anteriores.

A solução será trocar o Lambda por um EC2, que permite armazenamento persistente usando EBS (Elastic Block Store) ou discos locais.

Tendo em vista essas limitações, vamos criar nossa primeira função Lambda!

Abra o AWS Academy e acesse o Learner Lab. Inicialmente, busque por Lambda na guia de serviços:

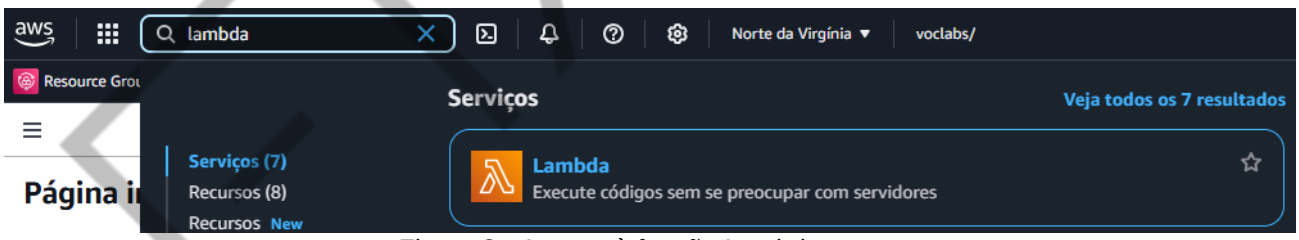

Figura 2 - Acesso à função Lambda
Fonte: Elaborado pelo autor (2024)

Lembre-se de selecionar a região “Norte da Virgínia” para a criação de suas funções, para que depois seja possível encontrá-las.

Em seguida, clique no menu Lambda > Funções e selecione o botão “Criar função”:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

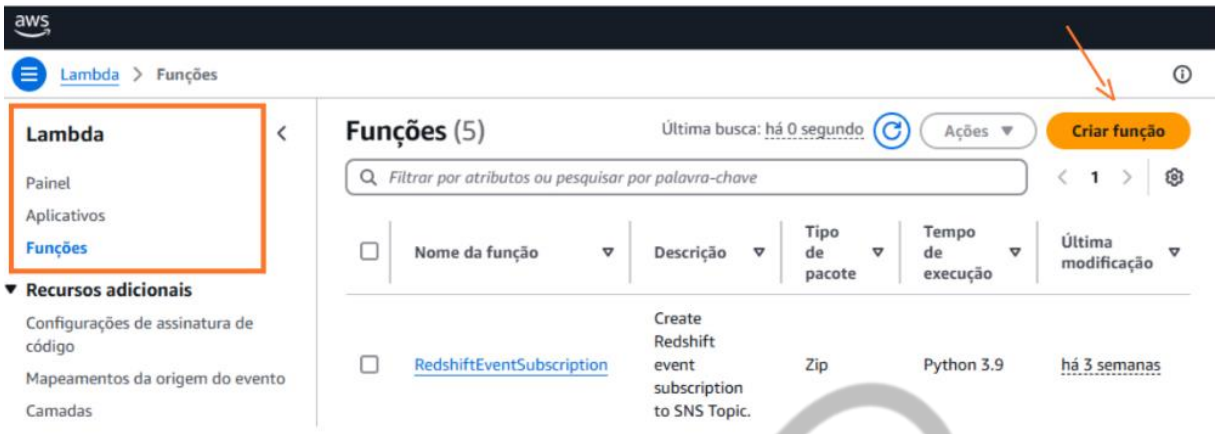

Figura 3 - Criando uma nova função Lambda
Fonte: Elaborado pelo autor (2024)

Defina o nome da função Lambda e o tempo de execução (*runtime*). No tempo de execução, também é importante definirmos a linguagem de programação que utilizaremos ou o *framework*/*engine* necessário para rodar nosso código. Neste caso, escolheremos o Python.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

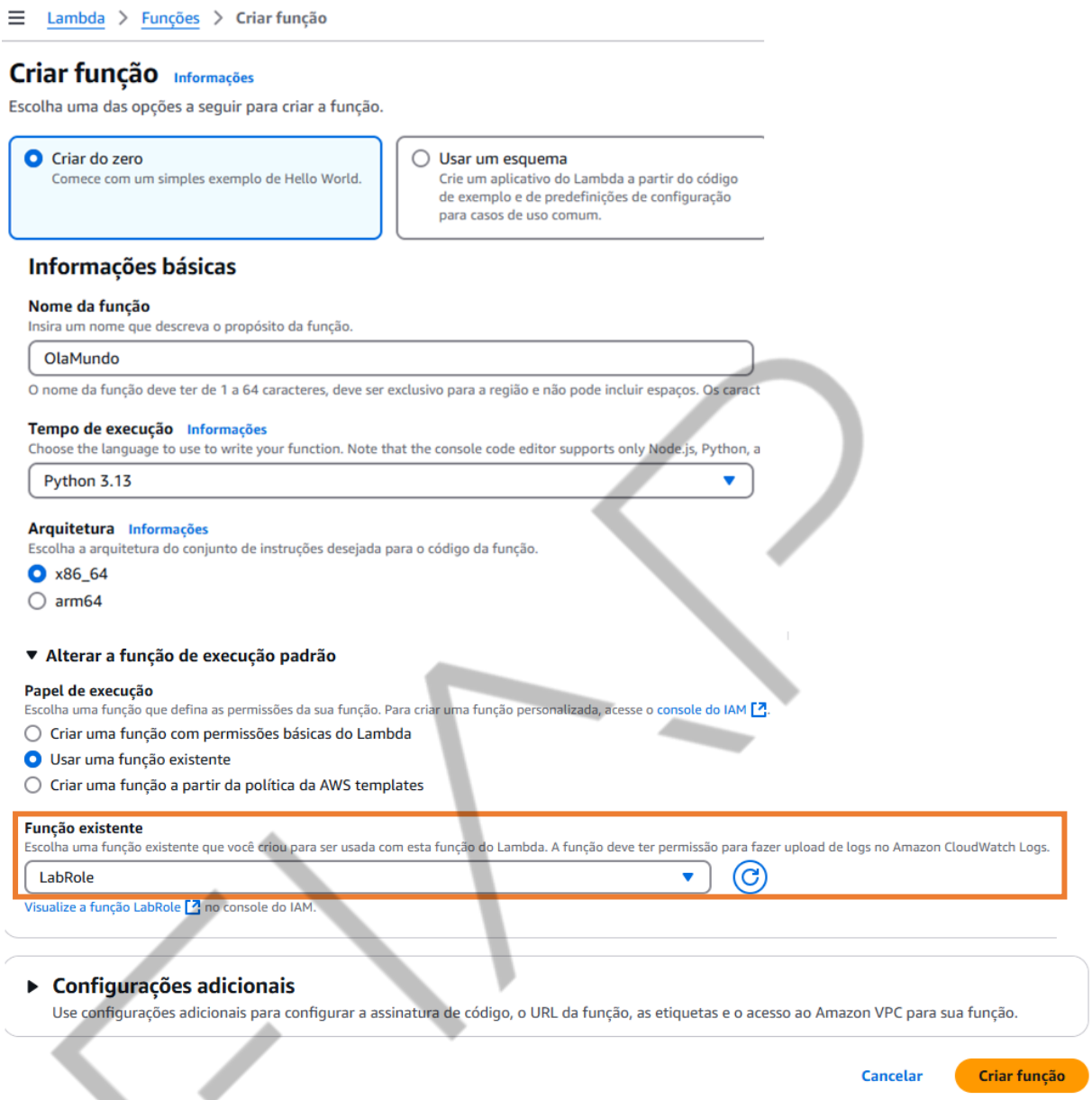

Figura 4 - Informações da função Lambda
Fonte: Elaborado pelo autor (2024)

Expanda a opção de “Alterar a função de execução de execução padrão” e selecione a opção “Usar uma função existente”. Como estamos utilizando o ambiente de testes (*Learner Lab*), é importante escolher a “LabRole” (*role*) para conseguir executar a função. Finalmente, clique em “Criar função”.

Ao criar a função, você visualizará um trecho de código na linguagem que selecionou (neste caso, Python) e já poderá testar a função.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

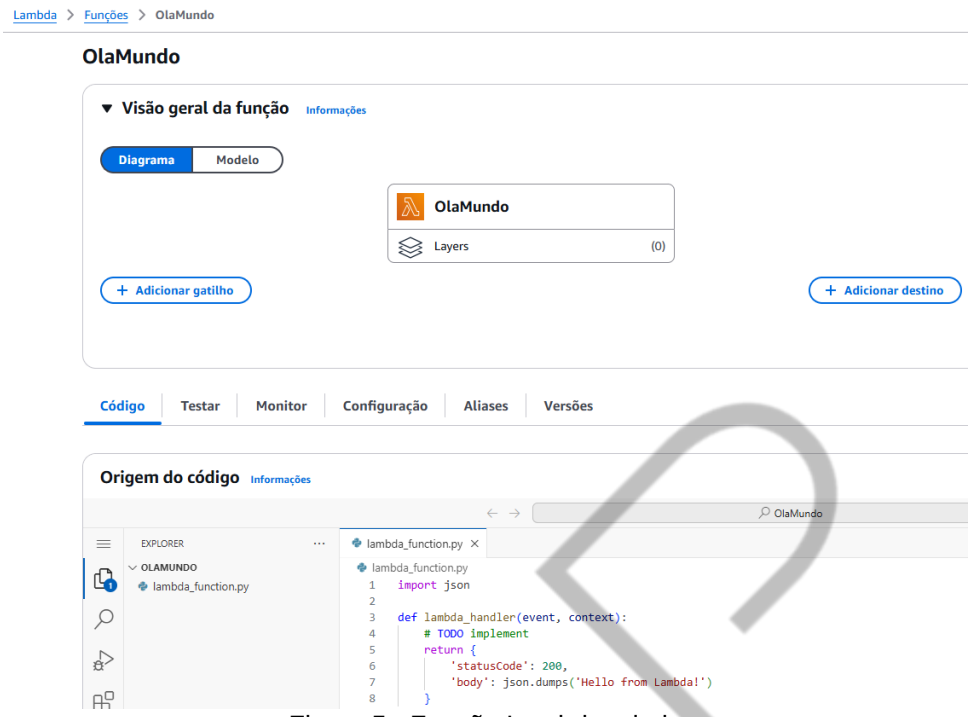

Figura 5 - Função Lambda criada
Fonte: Elaborado pelo autor (2024)

A seguir, iremos implantar a função Lambda e testá-la.

2.2 Testes da função Lambda

Inicialmente iremos alterar o texto da função Lambda, de acordo com o seguinte trecho de código:

```
import json

def lambda_handler(event, context):
    # TODO implement
    return {
        'statusCode': 200,
        'body': json.dumps('Ola FIAP ON!')
    }
```

Código-fonte 1 - Função Lambda básica
Fonte: Elaborado pelo autor (2024)

O que esse código está fazendo? O AWS Lambda recebe um evento de entrada (*event*) e um contexto (*context*) como entrada. O “event” contém os dados que acionaram a função (como uma requisição HTTP, um evento do S3, SNS, entre

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

outros). O “contexto” contém informações do ambiente de execução do Lambda. A função retorna um dicionário com dois campos principais:

- 'statusCode': 200 → Indica que a requisição foi bem-sucedida.
- 'body': json.dumps('Ola FIAP ON!') → Retorna a string "Ola FIAP ON!" no formato JSON.

```
{
    "statusCode": 200,
    "body": "\"Ola FIAP ON!\""
}
```

Código-fonte 2 - Exemplo de resposta se a função foi acionada
Fonte: Elaborado pelo autor (2024)

Isso significa que, ao ser chamada via API Gateway, por exemplo, ela responderá com um código HTTP 200 e o corpo da resposta conterá a string "Ola FIAP ON!" em JSON.

Toda alteração em uma função Lambda requer que a função seja reimplantada (*deploy*). Portanto, selecione a opção “Deploy” e, em seguida, clique no botão “Test”.

Você irá nomear um evento de teste, como “TesteOlaAlunos”. É possível ainda customizar um arquivo JSON de entrada para o evento. No entanto, mantenha as opções padrão, conforme ilustra a figura a seguir, e clique em “Save”.

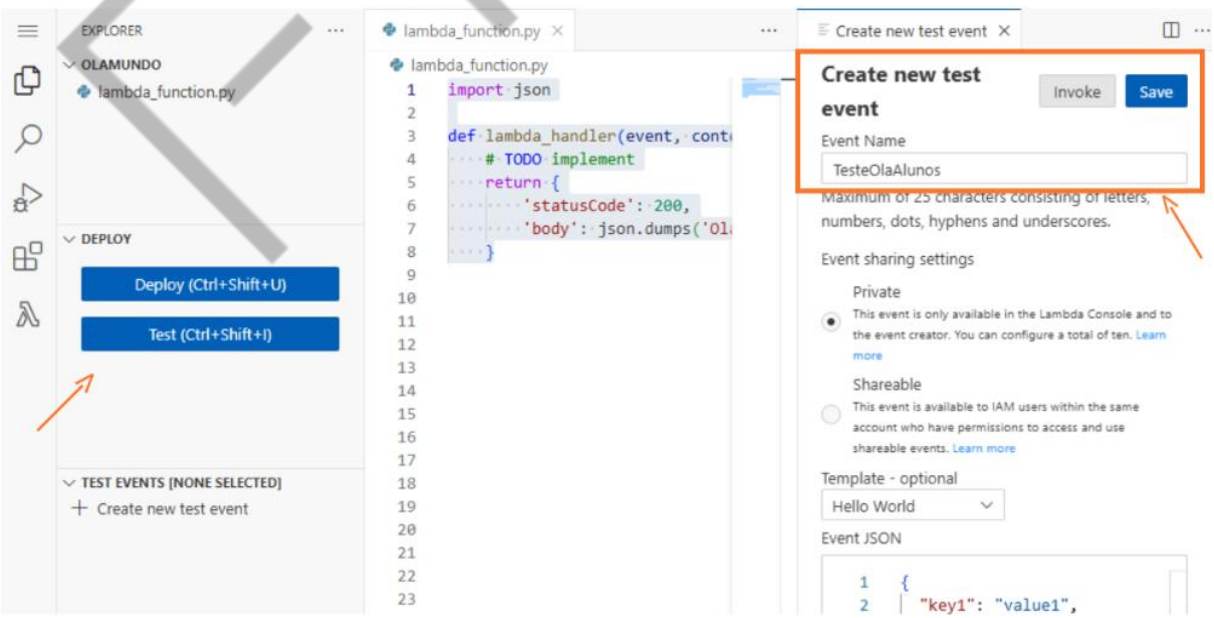

Figura 6 - Deploy e Teste de uma função Lambda
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Clique novamente no botão “Test” para executar a função Lambda.

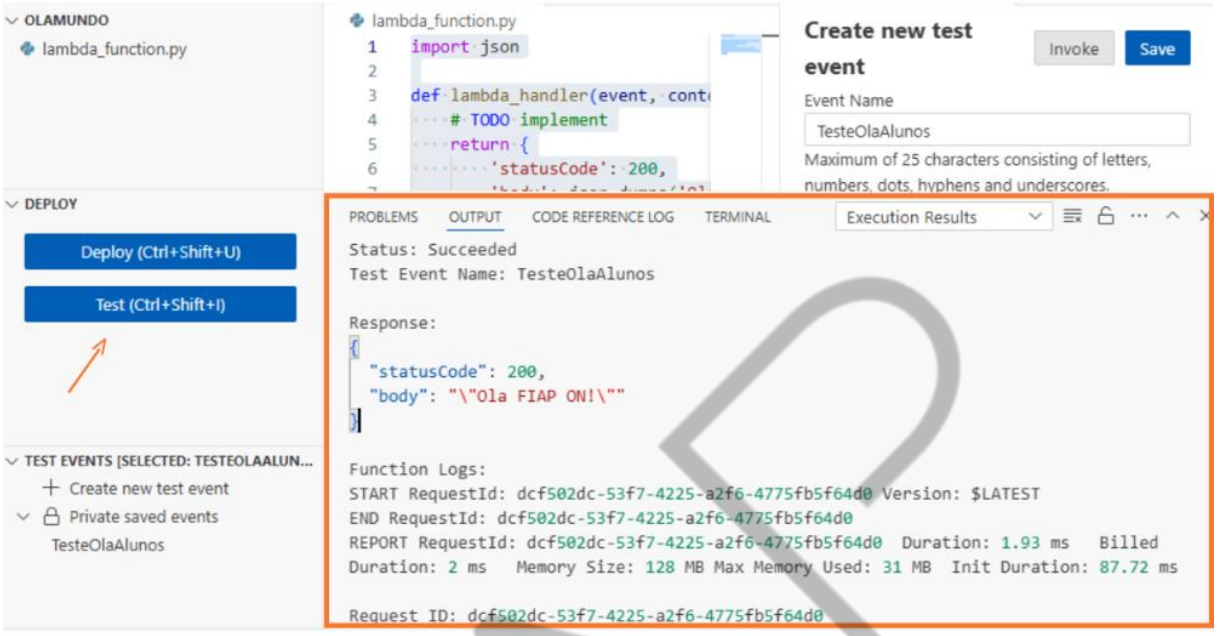

Figura 7 - Teste da Função Lambda
Fonte: Elaborado pelo autor (2024)

Note que é possível visualizar o resultado da execução, além do tempo de duração e a alocação de memória. Estas informações são contabilizadas para cobrança da função.

Agora que entendemos como funciona uma função Lambda, podemos aplicar em nosso cenário.

2.3 Transformação de dados com funções Lambda

Iremos implementar a primeira etapa da nossa solução completa. Para tanto, iremos criar funções Lambda que serão acionadas automaticamente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

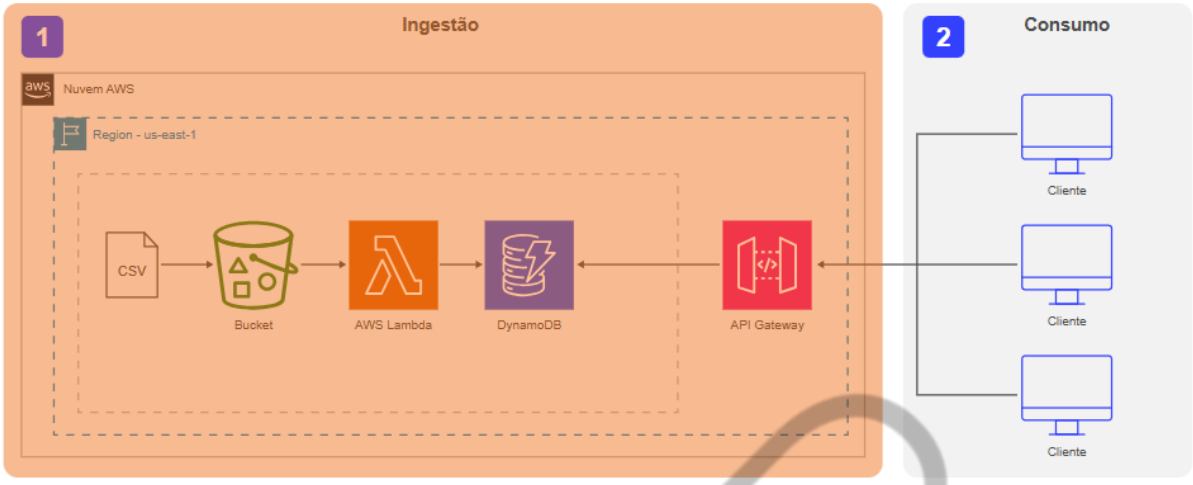

Figura 8 - Arquitetura da solução, em destaque laranja, tem-se a ingestão de Dados
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

Na figura anterior, em laranja, focamos na primeira parte da solução que é a ingestão de dados. Nesta parte, os dados são transformados para serem carregados em um banco de dados.

O banco de dados DynamoDB foi escolhido por ser uma base de dados *serverless*, ou seja, está alinhado com os requisitos da nossa solução de não nos preocuparmos com as instalações. No entanto, ele não suporta ingestão de arquivos por CSV, e sim, por meio de arquivos JSON. Portanto, teremos que transformar os dados inicialmente estruturados do CSV no formato JSON. A seguir, observe o detalhamento da solução a ser implementada.

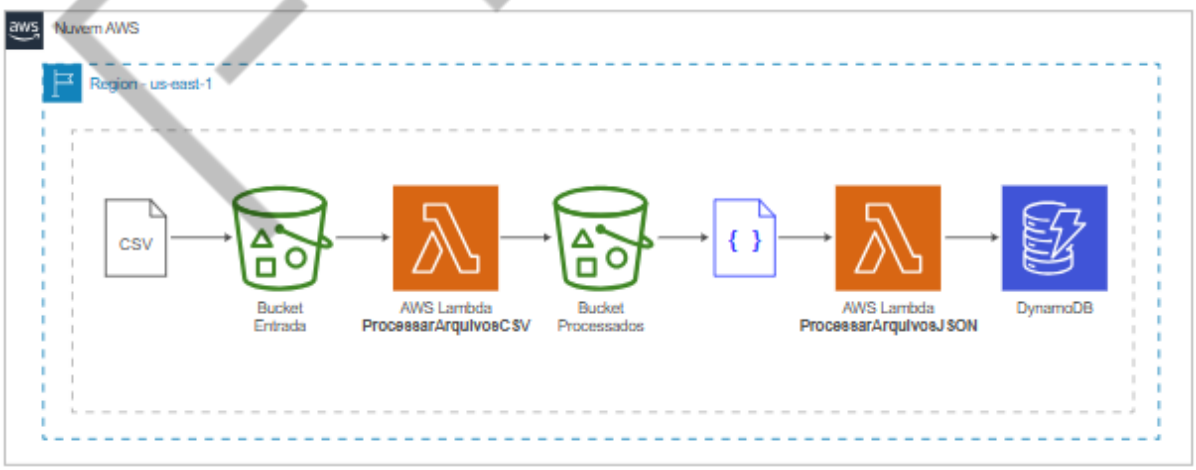

Figura 9 - Detalhamento da Solução
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na figura “Detalhamento da Solução”, podemos observar uma sequência de transformações sofridas por um arquivo de entrada no formato CSV. O arquivo é carregado inicialmente em um bucket de entrada do S3.

Em seguida, uma função Lambda denominada “ProcessarArquivosCSV” irá processar o arquivo CSV e transformá-lo em um formato JSON para que possa alimentar nosso banco de dados DynamoDB.

Finalmente, uma outra função Lambda denominada “ProcessarArquivosJSON” irá ler o arquivo JSON e inserir os dados no banco de dados DynamoDB. O processo inteiro será acionado **automaticamente** por meio de **gatilhos**. Vejamos a seguir como configurar esses gatilhos.

2.4 Disparo automático de funções Lambda (*triggers*)

A função Lambda pode ser executada automaticamente quando ocorre algum evento na AWS, como, por exemplo, a carga de dados em um bucket S3. Essa execução é chamada de “gatilho” ou *trigger.*

Para simular este processamento automático, iremos criar dois buckets S3 com o padrão:

- fiapon-<suasiniciais>-arquivos-entrada
- fiapon-<suasiniciais>-arquivos-processados

Altere o bucket de origem e destino para suas iniciais. Por exemplo, considerando que meu nome é Fernando Timóteo Fernandes, e também o padrão sugerido “fiapon-<suasiniciais>-arquivos-entrada”, temos o seguinte bucket: “fiapon-ftf-arquivos-entrada”.

Após a criação dos buckets, você deverá ter os seguintes buckets, lembrando que o nome dos buckets deverá refletir suas iniciais:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

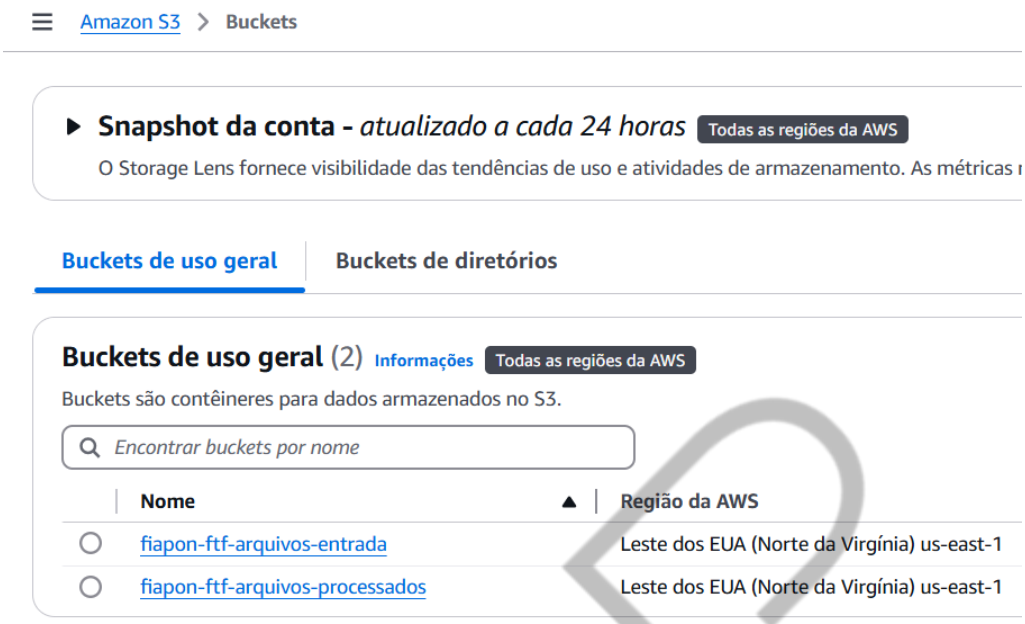

Figura 10 - Buckets criados
Fonte: Elaborado pelo autor (2024)

Agora, vamos agora criar uma função Lambda chamada "**ProcessarArquivosCSV**" para ler o arquivo a ser enviado ao bucket de entrada. Adapte o seguinte código, alterando o nome dos buckets de acordo com suas iniciais:

```
import boto3
import csv
import json
import os
from datetime import datetime

# Buckets
bucket_origem = 'fiapon-<suasiniciais>-arquivos-entrada'
bucket_destino = 'fiapon-<suasiniciais>-arquivos-processados'

# Função para processar um único arquivo do bucket de origem e retorna uma lista
def processar_arquivo(bucket_origem, nome_arquivo, s3_client):
    resultados = []
    try:
        print(f"Processando arquivo: {nome_arquivo}")

        # Ler o arquivo do S3
        arquivo_obj = s3_client.get_object(Bucket=bucket_origem, Key=nome_arquivo)
        arquivo_conteudo = arquivo_obj['Body'].read().decode('utf-8').splitlines()

        # Processar o CSV
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
            leitor_csv = csv.DictReader(arquivo_conteudo)

            for linha in leitor_csv:
                try:
                                                    temperatura     =
int(float(linha.get("temperatura", 0)))
                except ValueError:
                    temperatura = 0

                resultados.append({
                   "id_estacao": linha.get("id_estacao", ""),
                    "nome_arquivo": nome_arquivo,
                    "data": linha.get("data", ""),
                    "latitude": linha.get("latitude", ""),
                    "longitude": linha.get("longitude", ""),
                    "temperatura": temperatura
                })

            # Excluir o arquivo de origem após o processamento
                s3_client.delete_object(Bucket=bucket_origem,
Key=nome_arquivo)
            print(f"Arquivo {nome_arquivo} excluído do bucket
de origem.")

        except Exception as e:
                    print(f"Erro  ao  processar  o  arquivo
{nome_arquivo}: {e}")
            resultados.append({"nome_arquivo": nome_arquivo,
"erro": str(e)})

        return resultados

    # Função Lambda principal
    def lambda_handler(event, context):
        # Inicializar o cliente S3
        s3 = boto3.client('s3')

        try:
            # Recuperar o nome do arquivo do evento
            registro = event['Records'][0]
            nome_arquivo = registro['s3']['object']['key']

            # Processar o arquivo
              resultados  =  processar_arquivo(bucket_origem,
nome_arquivo, s3)

            # Gerar o JSON com os resultados
            if resultados:
                                                    timestamp       =
datetime.utcnow().strftime('%Y_%m_%d_%H_%M_%S')
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                                        nome_arquivo_base     =
os.path.splitext(nome_arquivo.split('/')[-1])[0]
                                        nome_arquivo_json     =
f"{nome_arquivo_base}_processado_{timestamp}.json"

                # Salvar o JSON no bucket de destino
                    json_resultados  =  json.dumps(resultados,
indent=4)
                s3.put_object(
                    Bucket=bucket_destino,
                    Key=nome_arquivo_json,
                    Body=json_resultados,
                    ContentType='application/json'
                )
                    print(f"Arquivo  JSON  {nome_arquivo_json}
gerado no bucket {bucket_destino}")

            # Retornar JSON de sucesso
            return {
                "statusCode": 200,
                "body": {
                        "mensagem":  "Arquivo  processado  com
sucesso!",
                    "arquivo_processado": nome_arquivo,
                    "resultado_arquivo": nome_arquivo_json
                }
            }

        except KeyError as e:
            print(f"Erro: parâmetro ausente - {e}")
            return {
                "statusCode": 400,
                "body": {
                   "mensagem": "Erro: Nome do arquivo ausente
no evento."
                }
            }
        except Exception as e:
            print(f"Erro geral ao processar o arquivo: {e}")
            return {
                "statusCode": 500,
                "body": {
                    "error": str(e)
                }
            }
```

Código-fonte 3 - Função Lambda que processa arquivos CSV
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Explicando sucintamente o que esse código faz, ele implementa uma **função AWS Lambda** para processar arquivos CSV armazenados no **Amazon S3**. Isto é, ele automatiza a leitura, transformação e movimentação de arquivos entre **dois buckets S3**, além de gerar um relatório JSON com os dados processados. Seu fluxo possui cinco etapas:

1. **Evento de Upload no S3:**
   - A função é acionada automaticamente sempre que um arquivo é enviado ao bucket de entrada (fiapon-<suasiniciais>-arquivos-entrada).
   - O evento fornece o nome do arquivo recém-carregado.
2. **Processamento do Arquivo CSV:**
   - A função lê o arquivo CSV do S3, decodificando seu conteúdo.
   - Usa a biblioteca csv.DictReader para transformar as linhas do CSV em um formato dicionário.
   - Converte a temperatura para um número inteiro, garantindo que valores inválidos não causem falhas.
   - Extrai dados importantes: ID da estação, data, latitude, longitude e temperatura.
3. **Exclusão do Arquivo Original:**
   - Após o processamento, o arquivo é excluído do bucket de origem, garantindo que ele não seja processado novamente.
4. **Geração de JSON com os Resultados:**
   - Um arquivo JSON contendo os dados processados é criado com um timestamp UTC no nome.
   - Esse JSON é salvo no bucket de saída (fiapon-<suasiniciais>-arquivos-processados).
5. **Retorno da Função:**
   - Se tudo der certo, a função retorna um **status 200**, indicando sucesso.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- Caso ocorra um erro (como um arquivo corrompido ou erro de permissão no S3), um código de erro apropriado (400 ou 500) é retornado, junto com a descrição do problema.

Exemplo de entrada e saída:

- **Entrada (Evento S3):**

Quando um arquivo estacao_01.csv é enviado ao **bucket de entrada**, a função Lambda é acionada automaticamente.

```
{
  "Records": [
    {
      "s3": {
        "object": {
          "key": "estacao_01.csv"
        }
      }
    }
  ]
}
```

Código-fonte 4 - Exemplo de arquivo CSV sendo carregado na entrada para ser convertido para JSON
Fonte: Elaborado pelo autor (2024)

Exemplo de CSV no S3:

```
id_estacao,data,latitude,longitude,temperatura
123,2024-02-01,-23.55,-46.63,28.5
456,2024-02-02,-22.90,-43.17,30.0
```

Código-fonte 5 - Exemplo de conteúdo do CSV
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **Saída (JSON gerado no Bucket de Destino):**

```
[
  {
    "id_estacao": "123",
    "nome_arquivo": "estacao_01.csv",
    "data": "2024-02-01",
    "latitude": "-23.55",
    "longitude": "-46.63",
    "temperatura": 28
  },
  {
    "id_estacao": "456",
    "nome_arquivo": "estacao_01.csv",
    "data": "2024-02-02",
    "latitude": "-22.90",
    "longitude": "-43.17",
    "temperatura": 30
  }
]
```

Código-fonte 6 - Exemplo de saída após a conversão para JSON
Fonte: Elaborado pelo autor (2024)

Dando sequência, acesse a função Lambda “ProcessarArquivosCSV” e crie um gatilho para o bucket de entrada, clicando no botão “Adicionar gatilho”.

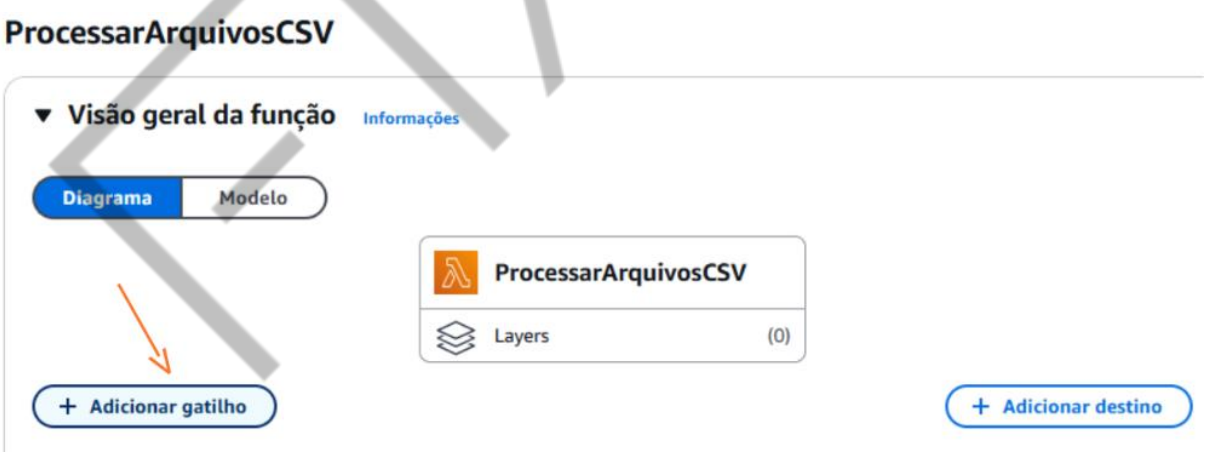

Figura 11 - Criar gatilho
Fonte: Elaborado pelo autor (2024)

Agora, configure o gatilho, selecionando o bucket recém-criado de entrada no campo “Bucket”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

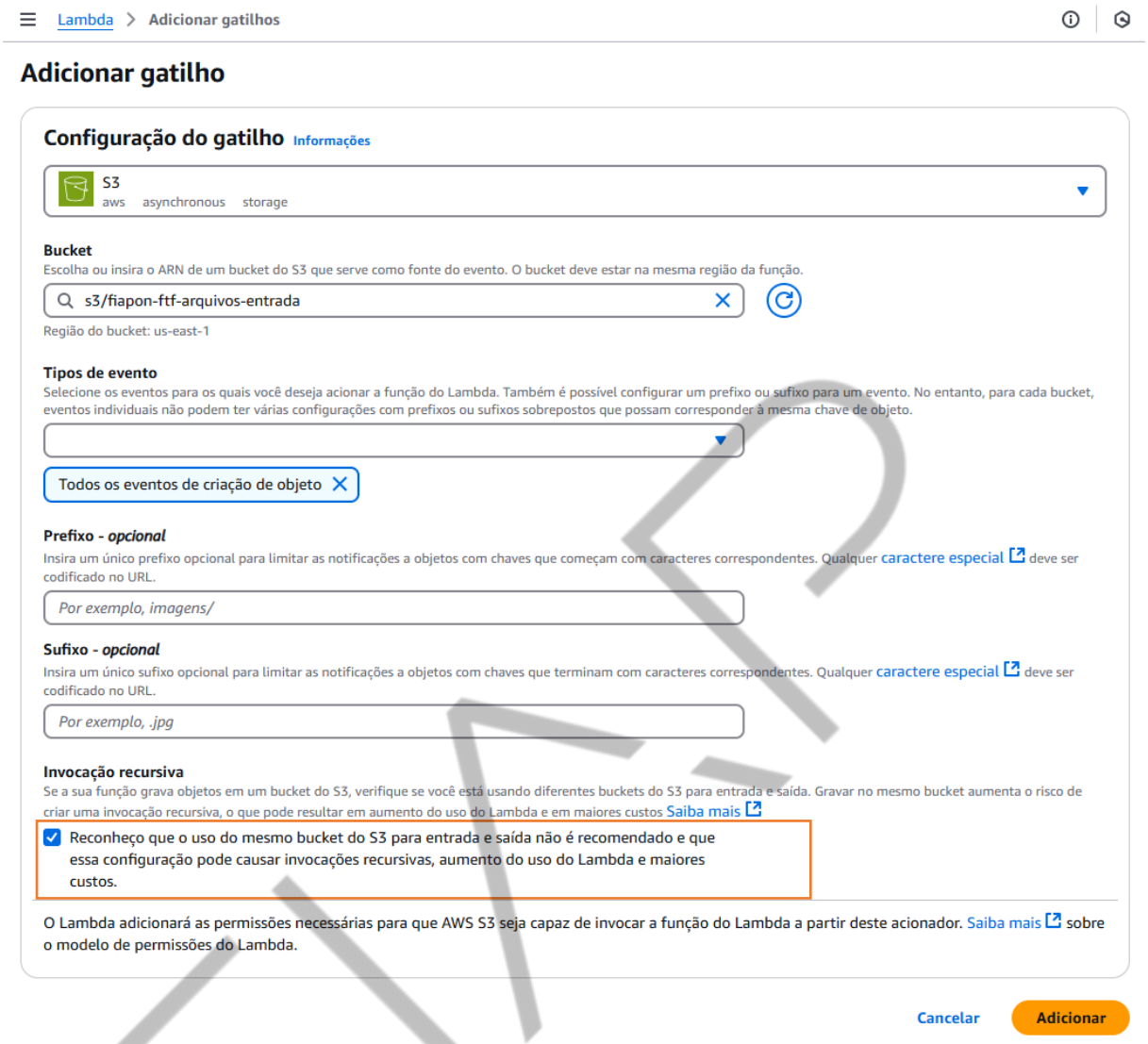

Figura 12 - Gatilho para o bucket S3
Fonte: Elaborado pelo autor (2024)

Importante: não esqueça de marcar a caixa de texto de “invocação recursiva”.

Agora basta criar dois arquivos de testes e subi-los ao bucket de entrada, conforme tabela a seguir.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Nome do arquivo: **estacao_1_2024_12_20.csv**	**Nome do arquivo:** **estacao_1_2024_12_21.csv**
id_estacao,data,latitude,longitude,temperatura	id_estacao,data,latitude,longitude,temperatura
1,2024-12-20 08:00:00,-23.5505,-46.6333,22.3	1,2024-12-21 08:00:00,-23.5505,-46.6333,23.5
1,2024-12-20 09:00:00,-23.5505,-46.6333,24.1	1,2024-12-21 09:00:00,-23.5505,-46.6333,25.2
1,2024-12-20 10:00:00,-23.5505,-46.6333,30.2	1,2024-12-21 10:00:00,-23.5505,-46.6333,31.0
1,2024-12-20 11:00:00,-23.5505,-46.6333,27.8	1,2024-12-21 11:00:00,-23.5505,-46.6333,28.5
1,2024-12-20 12:00:00,-23.5505,-46.6333,29.4	1,2024-12-21 12:00:00,-23.5505,-46.6333,30.0
1,2024-12-20 13:00:00,-23.5505,-46.6333,31.0	1,2024-12-21 13:00:00,-23.5505,-46.6333,32.1
1,2024-12-20 14:00:00,-23.5505,-46.6333,25.7	1,2024-12-21 14:00:00,-23.5505,-46.6333,26.3
1,2024-12-20 15:00:00,-23.5505,-46.6333,26.9	1,2024-12-21 15:00:00,-23.5505,-46.6333,22.8
1,2024-12-20 16:00:00,-23.5505,-46.6333,27.1	1,2024-12-21 16:00:00,-23.5505,-46.6333,23.4
1,2024-12-20 17:00:00,-23.5505,-46.6333,26.8	1,2024-12-21 17:00:00,-23.5505,-46.6333,24.5
1,2024-12-20 18:00:00,-23.5505,-46.6333,25.4	1,2024-12-21 18:00:00,-23.5505,-46.6333,24.3

Tabela 1 - Conteúdo dos arquivos de teste
Fonte: Elaborado pelo autor (2024)

Na tabela anterior, exibimos o conteúdo dos arquivos de testes para que você consiga criar arquivos CSV simulando dados dos sensores. No primeiro arquivo, simulamos dados de uma estação meteorológica, com informações de temperaturas na região de São Paulo, das 08:00 da manhã até às 18:00. No segundo arquivo, simulamos dados da mesma estação no dia posterior.

Agora vamos subir os arquivos ao bucket de entrada.

Figura 13 - Acesso ao bucket de entrada de dados
Fonte: Elaborado pelo autor (2024)

Acesse o bucket de entrada e clique em carregar para adicionar um arquivo ao bucket.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

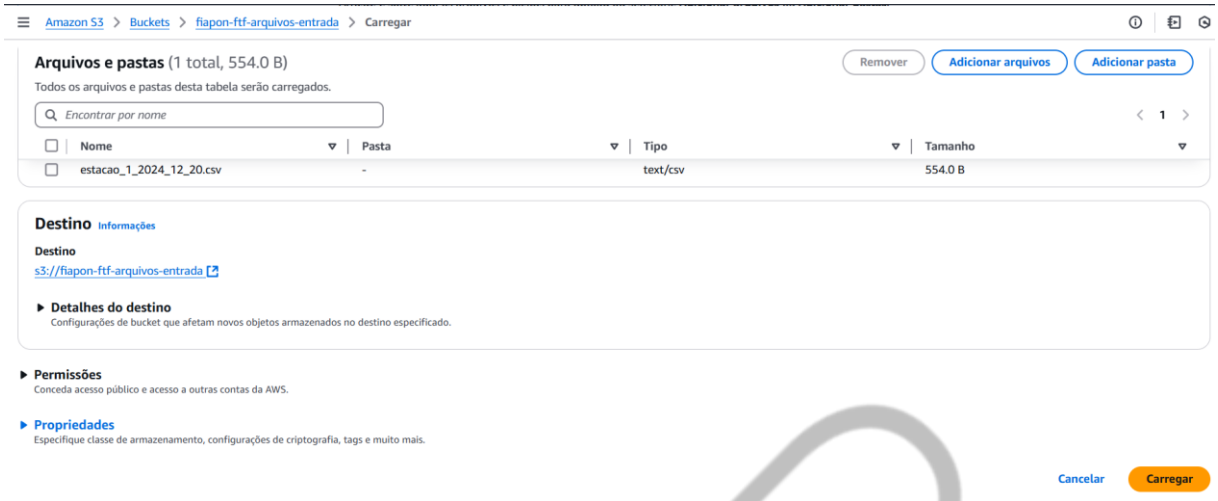

Figura 14 - Carga do arquivo CSV no bucket
Fonte: Elaborado pelo autor (2024)

Depois de alguns minutos (espere até 5 minutos), você verá que foi gerado um arquivo JSON no bucket de arquivos processados. Além disso, o arquivo CSV será excluído do bucket de entrada após o processamento concluído.

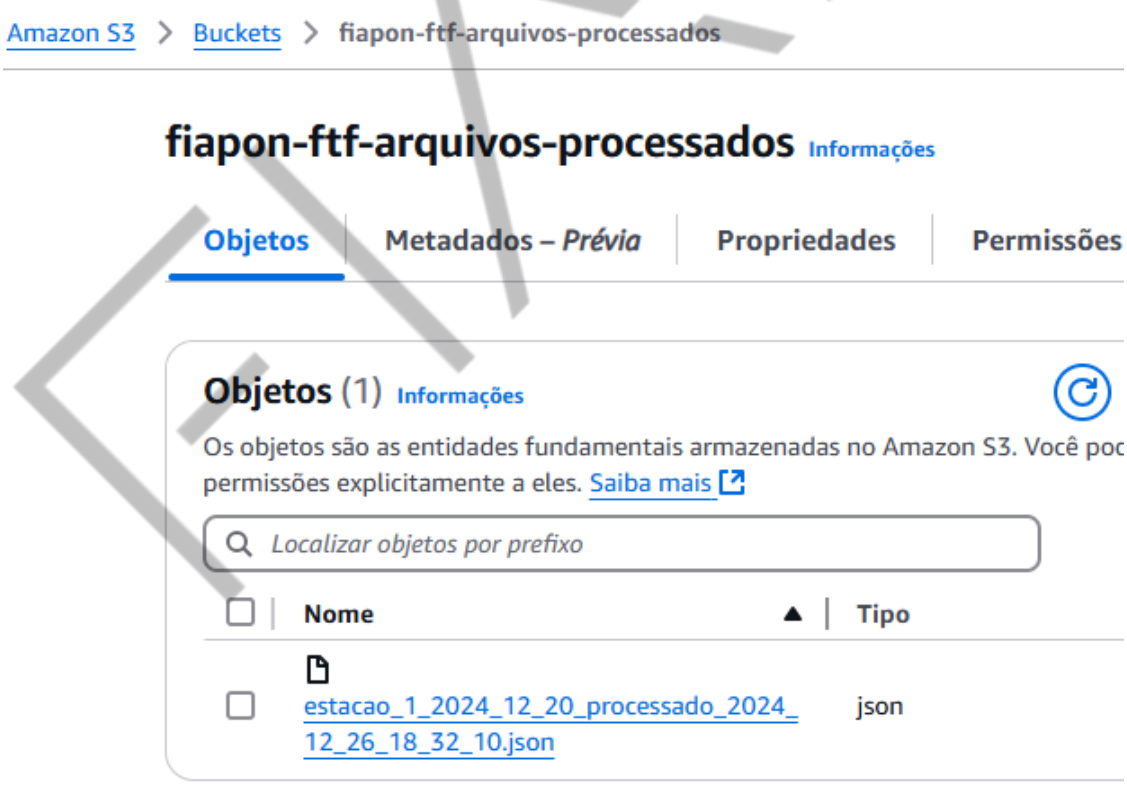

Figura 15 - Arquivo processado
Fonte: Elaborado pelo autor (2024)

Os arquivos serão processados automaticamente toda vez que você subir um arquivo para o bucket de entrada. A seguir, exibimos parte do arquivo JSON gerado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
[
    {
        "id_estacao": "1",
        "nome_arquivo": "estacao_1_2024_12_20.csv",
        "data": "2024-12-20 08:00:00",
        "latitude": "-23.5505",
        "longitude": "-46.6333",
        "temperatura": 22
    },
    {
        "id_estacao": "1",
        "nome_arquivo": "estacao_1_2024_12_20.csv",
        "data": "2024-12-20 09:00:00",
        "latitude": "-23.5505",
        "longitude": "-46.6333",
        "temperatura": 24
    },
```


Figura 16 - Conteúdo do arquivo JSON gerado
Fonte: Elaborado pelo autor (2024)

Finalizamos a primeira parte da ingestão de dados, fazendo a transformação de dados para que possam ser gravados no banco de dados. Agora veremos como criar uma função Lambda para gravar o arquivo JSON gerado no DynamoDB.

2.5 Ingestão de dados com funções Lambda

Nesta etapa, vamos criar uma outra função Lambda que fará a leitura dos arquivos JSON gerados e gravará em uma tabela do DynamoDB.

Para isso, crie a função “**ProcessarArquivosJSON**” com o seguinte conteúdo. Note que a parte em destaque (bucket_json = 'fiapon-<**suasiniciais**>-arquivos-processados') é para que você altere o nome do bucket com suas iniciais.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import boto3
import json
import uuid
import logging

# Inicialização dos clientes AWS
s3_client = boto3.client('s3')
dynamodb = boto3.resource('dynamodb')
table = dynamodb.Table('temperaturas')

logger = logging.getLogger()
logger.setLevel(logging.INFO)

bucket_json = 'fiapon-<suasiniciais>-arquivos-processados'


def lambda_handler(event, context):
    try:
        # Obtendo informações do evento do S3
        file_key = event['Records'][0]['s3']['object']['key']

        logger.info(f"Processando arquivo: {file_key} do bucket: {bucket_json}")

        # Baixar o arquivo JSON do bucket
        response = s3_client.get_object(Bucket=bucket_json, Key=file_key)
        file_content = response['Body'].read().decode('utf-8')

        # Parse do conteúdo do arquivo JSON
        data = json.loads(file_content)

        # Inserir cada item no DynamoDB
        for item in data:
            table.put_item(
                Item={
                    'id': str(uuid.uuid4()),  # Gerar um UUID único
                    'data': item['data'],
                    'id_estacao': item['id_estacao'],
                    'nome_arquivo': item['nome_arquivo'],
                    'latitude': item['latitude'],
                    'longitude': item['longitude'],
                    'temperatura': item['temperatura']
                }
            )
            logger.info(f"Registro inserido no DynamoDB: {item}")
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                # Excluindo o arquivo do bucket após o processamento
                s3_client.delete_object(Bucket=bucket_json, Key=file_key)
                logger.info(f"Arquivo {file_key} excluído do bucket {bucket_json}")

            return {
                'statusCode': 200,
                'body': json.dumps(f"Arquivo {file_key} processado com sucesso.")
            }

        except Exception as e:
                logger.error(f"Erro ao processar arquivo: {str(e)}")
            return {
                'statusCode': 500,
                'body': json.dumps(f"Erro ao processar arquivo: {str(e)}")
            }
```

Código-fonte 7 - Função Lambda para gravar dados no DynamoDB
Fonte: Elaborado pelo autor (2024)

Explicando rapidamente o objetivo do código anterior, ele implementa uma função AWS Lambda que processa arquivos JSON armazenados no Amazon S3 e insere seus dados em uma tabela do DynamoDB. Ele é acionado automaticamente sempre que um novo arquivo JSON é carregado no bucket S3 definido.

Primeiramente, a função obtém o nome do arquivo a partir do evento de *trigger* do S3. Em seguida, faz o download do JSON, lê seu conteúdo e converte para um formato Python utilizando json.loads(). Assim, cada registro do JSON é inserido no DynamoDB, onde recebe um UUID único como identificador.

Após a inserção bem-sucedida dos dados na tabela, o arquivo JSON original é excluído do bucket S3 para evitar reprocessamento. Caso ocorra algum erro, a função registra a falha nos logs do **CloudWatch** e retorna um **código de erro HTTP 500**.

Dando sequência com o código anterior, preparamos a função Lambda para processar o arquivo JSON e inserir os registros no DynamoDB. No entanto, ainda não iremos executá-lo, pois iremos acioná-lo automaticamente por meio de uma API.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Finalizamos a **ingestão** de dados *serverless* utilizando as funções Lambda e o DynamoDB, completando, assim, a primeira parte da nossa solução.

Figura 17 - Ingestão de Dados
Fonte: Elaborado pelo autor (2024)

Vejamos a seguir como **consumir** estes dados por meio de uma API REST utilizando o serviço da AWS denominado **AWS API Gateway**.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

3 CONSUMO DE DADOS NA NUVEM

Neste momento, iremos implementar a segunda parte de nossa solução *serverless*, onde consumiremos os dados do DynamoDB por meio de uma API, como pode ser vista na arquitetura da solução, destacada em laranja na figura a seguir.

Uma API (Application Programming Interface) permite a troca de informações entre sistemas distintos, padronizando o formato do arquivo a ser trocado.

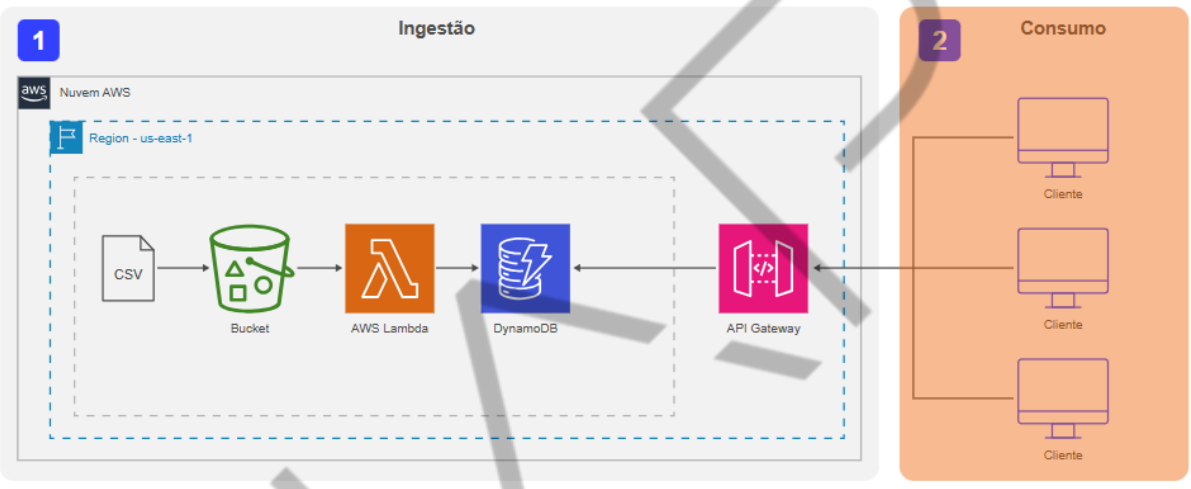

Figura 18 - Consumo de dados por meio de API, no destaque em laranja
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

A figura a seguir ilustra como uma API pode ser utilizada para troca de informações entre sistemas.

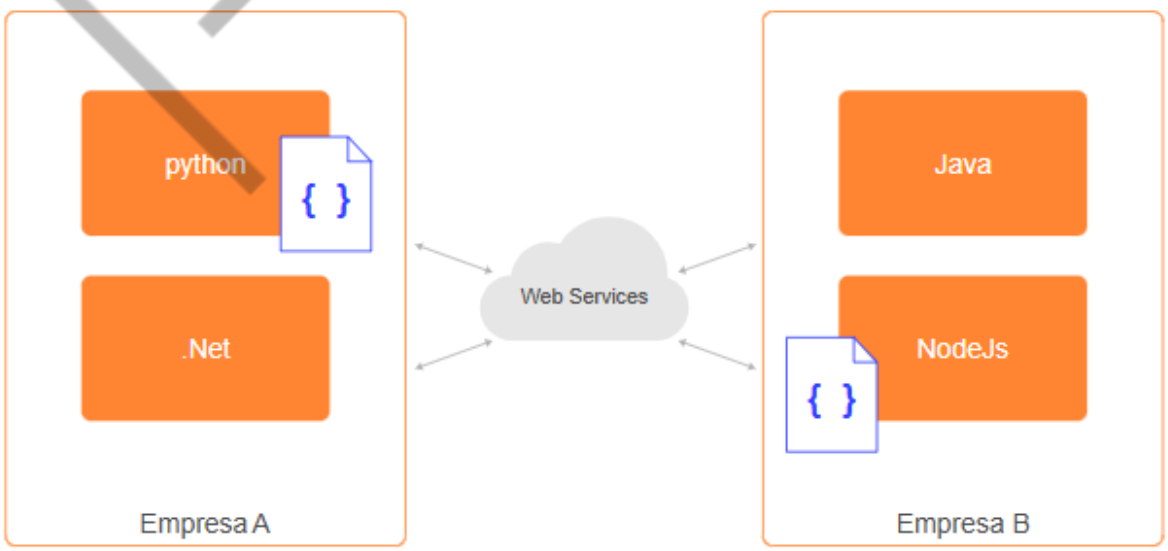

Figura 19 - Troca de informações por API
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Portanto, conforme figura “Troca de informações por API”, é possível consumir dados produzidos por sistemas com tecnologias distintas, desde que tais tecnologias estejam aptas a consumir os dados no formato especificado.

É possível utilizar diferentes tipos de formatos padronizados, como XML e JSON.

A AWS oferece a possibilidade de criação de APIs REST na nuvem, com o serviço denominado API Gateway. Primeiro vamos entender os conceitos básicos de uma API para então criarmos nossa API de consumo na AWS.

3.1 API Gateway

A AWS permite criar diferentes tipos de APIs, como:

- API WebSocket
- API HTTP
- API REST

Uma API WebSocket permite estabelecer um canal aberto para a troca de mensagens entre um cliente e um servidor, como a criação de um chat.

Uma API REST (Representational State Transfer) é um estilo arquitetural para construir serviços web que permitem a comunicação entre sistemas, geralmente usando o protocolo HTTP. Em uma API REST, cada operação é independente e os dados enviados não são reutilizados, o que comumente é descrito como o não mantenimento de estado entre as operações.

A API HTTP e a API REST são ambas RESTful, ou seja, não mantêm estado entre as operações. Elas diferem em relação às questões de custos e recursos de implementação.

Na API HTTP é possível utilizar a autenticação mais simples com o uso de JWT (JSON Web Token). O JWT é uma forma de autenticação que permite que um servidor verifique a identidade de um usuário sem precisar armazenar informações sobre ele.

A API REST não conta com autenticação por meio de JWT, mas conta com diversos outros recursos, como a criação de chaves de API e integração à serviços

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

de autenticação, como o Amazon Cognito. Também é possível estabelecer limites de chamadas por clientes, *caching* e validação de requisição antes de executar uma integração com o API REST.

A API HTTP se destaca por ter um tempo menor de resposta desde a requisição até a resposta a um recurso da API (latência) a um custo reduzido em comparação com a API REST. Cada recurso é identificado por uma URI (Uniform Resource Identifier) e por uma rota que acessa um recurso. O endereço base da API, seguido do nome de um recurso normalmente, é chamado de *endpoint*. Por exemplo: <https://api.exemplo.com/temperaturas>. Nesta URL, poderíamos consultar todas as temperaturas armazenadas em um banco de dados, a partir da integração com uma função Lambda.

É importante ressaltar que, ao criar uma API HTTP, é possível criar somente *endpoints* públicos. Já nas API REST, é possível criar tanto *endpoints* públicos como privados acessíveis somente dentro da VPC.

3.2 Métodos HTTP

As APIs RESTful seguem um conjunto de princípios e convenções para permitir operações de criação, leitura, atualização e exclusão de itens, por exemplo: Create, Read, Update e Delete — CRUD.

Os métodos HTTP são utilizados para realizar operações em recursos:

- **Método GET:** recuperar um recurso (exemplo: listar registros de temperaturas).
- **Método POST:** criar um novo recurso (exemplo: adicionar um novo registro).
- **Método PUT:** atualizar completamente um recurso existente (exemplo: editar um registro).
- **Método PATCH:** atualizar parcialmente um recurso existente.
- **Método DELETE:** remover um recurso (exemplo: excluir um registro).

Cada requisição HTTP pode retornar um dos seguintes status:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **1XX – Informativo**.
- **2XX – Sucesso**, por exemplo, HTTP 200 – OK; 201 – Criação de um registro; 204 – Exclusão bem sucedida.
- **3XX – Redirecionamento para outra URL**, por exemplo, Erro: 301 - Recurso Movido Permanentemente.
- **4XX – Erros na requisição do cliente**, por exemplo, Erro: 404 – página não encontrada.
- **5XX – Erros do Servidor**, por exemplo: Erro 500 – *Internal Server Error.*

O ideal é recebermos o retorno com o status 200, significando que a requisição foi bem-sucedida.

Agora que sabemos as características de uma API, iremos criar uma API que permite realizar consultas em um banco de dados DynamoDB.

3.3 Consumo da API

Vejamos em detalhes como iremos implementar esta solução usando uma API na AWS:

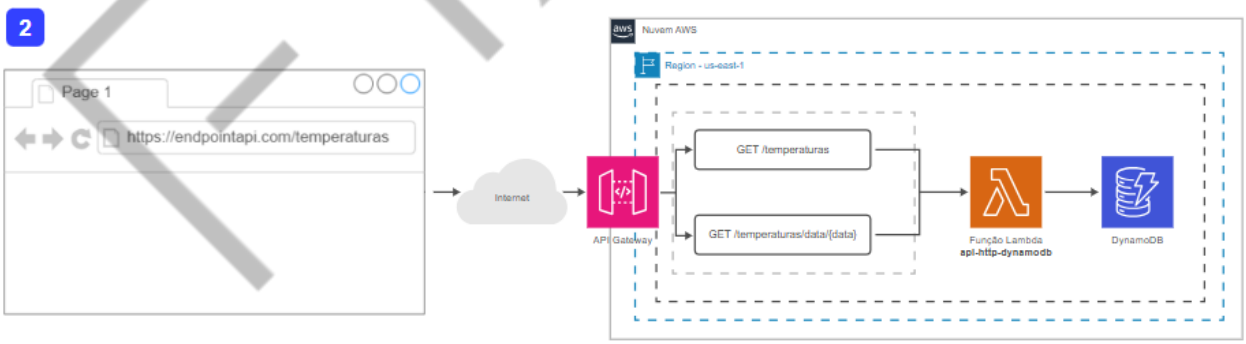

Figura 20 - Detalhamento da solução de consumo
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2025)

Na figura anterior, podemos visualizar um navegador web acessando uma URL. A URL irá apontar para nossa API, tanto para realizar consultas de todas as temperaturas, ou por uma data específica. Para isso, a API acionará uma função Lambda que irá ler os dados de temperaturas do DynamoDB.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Iremos utilizar uma única API para duas operações, uma de consulta geral e outra de consulta por data. A API acionará uma função Lambda para ler os dados do DynamoDB. Portanto, crie uma função Lambda com o nome "`api-http-dynamodb`" para processar as chamadas da API e consultar a tabela "temperaturas" do DynamoDB.

```
import json
import boto3
import uuid
from datetime import datetime
from boto3.dynamodb.conditions import Attr

client = boto3.client('dynamodb')
dynamodb = boto3.resource("dynamodb")
table = dynamodb.Table('temperaturas')
tableName = 'temperaturas'

def lambda_handler(event, context):
    print(event)
    body = {}
    statusCode = 200
    headers = {
        "Content-Type": "application/json"
    }

    try:
        if event['routeKey'] == "GET /temperaturas":
            body = table.scan()
            body = body["Items"]
            print("Medicoes----")
            print(body)
            body = sorted(body, key=lambda x: x['data'])
            responseBody = []
            for items in body:
                responseItems = {
                                                    'id_estacao':
int(items['id_estacao']),
                        'id': items['id'],
                        'data': items['data'],
                                                'temperatura':
int(items['temperatura']),
                      'latitude': float(items['latitude']),
                                                     'longitude':
float(items['longitude']),
                      'nome_arquivo': items['nome_arquivo']
                }
                responseBody.append(responseItems)
            body = responseBody
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                        elif    event['routeKey']    ==    "GET
/temperaturas/data/{data}":
                  data_param = event['pathParameters']['data']
                  try:
                      # Convert data_param to date
                                                  date_filter    =
datetime.strptime(data_param, "%Y-%m-%d").date()

                      # Filter by date ignoring timestamp
                      response = table.scan(
                          FilterExpression=Attr('data').begins
_with(str(date_filter))
                      )

                      responseBody = []

                      for items in response["Items"]:
                          responseItems = {
                                                      'id_estacao':
int(items['id_estacao']),
                                  'id': items['id'],
                                                     'temperatura':
int(items['temperatura']),
                                                       'latitude':
float(items['latitude']),
                                                      'longitude':
float(items['longitude']),
                                                   'nome_arquivo':
items['nome_arquivo'],
                                  'data': items['data']
                          }
                          responseBody.append(responseItems)

                     body = sorted(responseBody, key=lambda x:
x['data'])

                      print("Dados filtrados por data----")
                      print(body)

                  except ValueError:
                      statusCode = 400
                        body = f"Data invalida: {data_param}.
Formato esperado: YYYY-MM-DD"

          except KeyError:
              statusCode = 400
             body = 'Rota nao suportada: ' + event['routeKey']
          body = json.dumps(body)
          res = {
              "statusCode": statusCode,
              "headers": {
```

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
                "Content-Type": "application/json"
            },
            "body": body
        }
        return res
```

Código-fonte 8 - Função Lambda para consumo da API
Fonte: Elaborado pelo autor (2024)

Esse código-fonte implementa uma outra **função AWS Lambda** que expõe uma API para consultar medições de temperatura armazenadas no **Amazon DynamoDB**. Ele responde a requisições GET, permitindo recuperar todas as medições ou filtrar por uma data específica.

Quando a requisição **"GET /temperaturas"** é recebida, a função escaneia a tabela **"temperaturas"** no DynamoDB, ordena os registros pela data e retorna um JSON contendo informações como **ID da estação, data, temperatura, latitude** e **longitude**.

Se a requisição seguir o formato **"GET /temperaturas/data/{data}"**, a função extrai o parâmetro {data} da URL, converte para o formato de data (YYYY-MM-DD) e busca registros que começam com essa data. Os resultados são organizados e retornados em JSON.

Caso a rota solicitada não seja suportada ou a data esteja em formato inválido, a função responde com um **código de erro HTTP 400** e uma mensagem apropriada. Toda a comunicação é estruturada no formato JSON para facilitar a integração com serviços externos.

Agora que já temos mais uma função Lambda pronta, vamos criar a nossa API para responder às requisições em um *endpoint* público (um *endpoint* público é qualquer computador ou dispositivo que acesse o seu serviço de fora da AWS sem autenticação especial). Por exemplo:

<https://abcd1234.execute-api.us-east-1.amazonaws.com/prod/temperaturas>.

Para isso, busque por API na tela do console da AWS.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

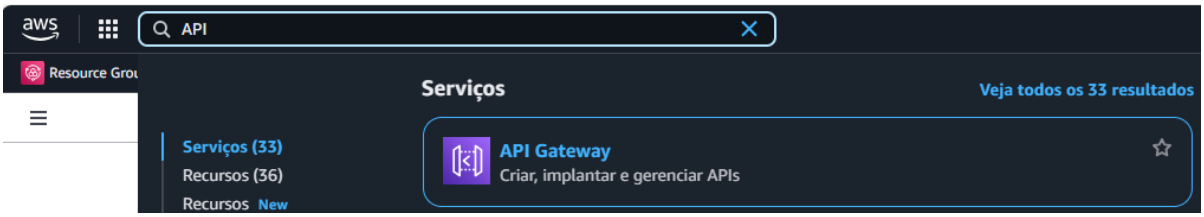

Figura 21 - Criação da API
Fonte: Elaborado pelo autor (2024)

Clique no botão “Compilar” dentro da opção de API HTTP.

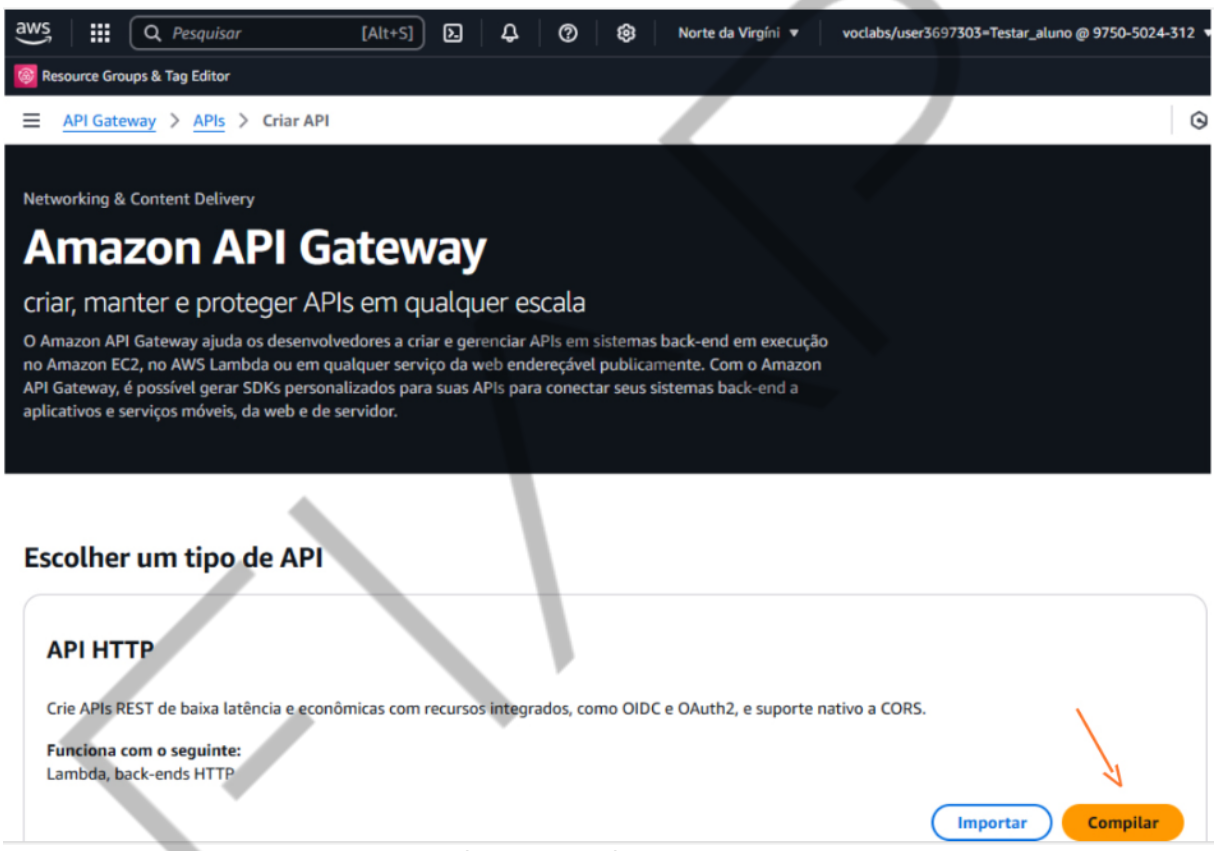

Figura 22 - Criar API HTTP
Fonte: Elaborado pelo autor (2024)

Defina o nome da API como “api-http-temperaturas” e avance sem criar nenhuma rota, mantendo as opções padrão (*default*) de estágio de implantação.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

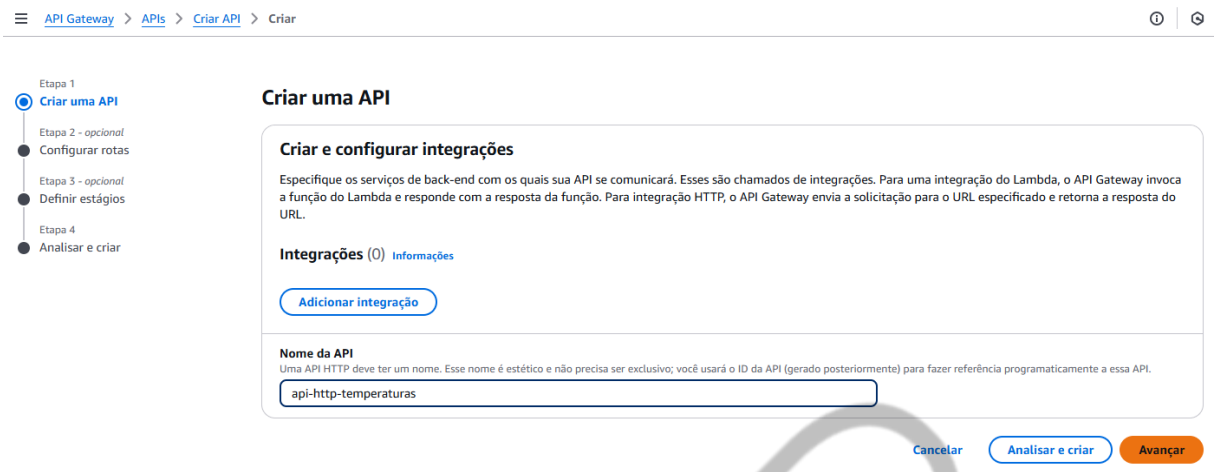

Figura 23 - Criação da API HTTP
Fonte: Elaborado pelo autor (2024)

Em seguida, clique em Rotas > Criar.

Figura 24 - Criação de Rotas
Fonte: Elaborado pelo autor (2024)

Inicialmente, existe uma opção para selecionar entre os métodos HTTP (GET, POST, PUT, DELETE). Selecione o método desejado e insira o caminho da rota. Veja em detalhes a criação de uma rota de exclusão:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

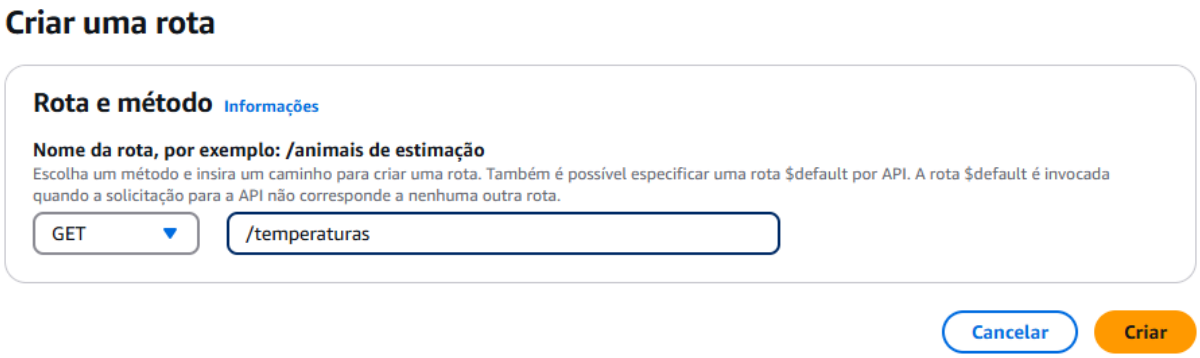

Figura 25 - Escolha do método HTTP e caminho da rota
Fonte: Elaborado pelo autor (2024)

Você poderá escolher o tipo do método HTTP e, em seguida, definir a rota para acesso a um recurso, conforme figura “Escolha do método HTTP e caminho da rota”.

Crie as seguintes rotas:

- **GET /temperaturas**
  - **Descrição:** rota para consultar todas as temperaturas armazenadas no DynamoDB.
- **GET / temperaturas/data/{data}**
  - **Descrição:** rota para consultar as temperaturas de um único dia especificado como parâmetro na chamada da API.

Após a criação, você terá esse cenário:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

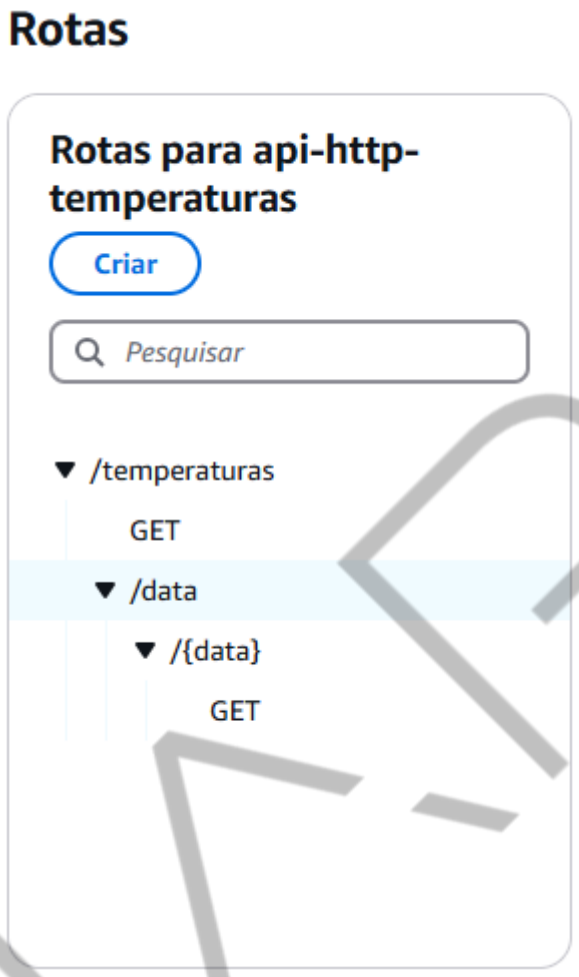

Figura 26 - Rotas criadas
Fonte: Elaborado pelo autor (2024)

Selecione a primeira rota e clique no botão “Anexar integração”.

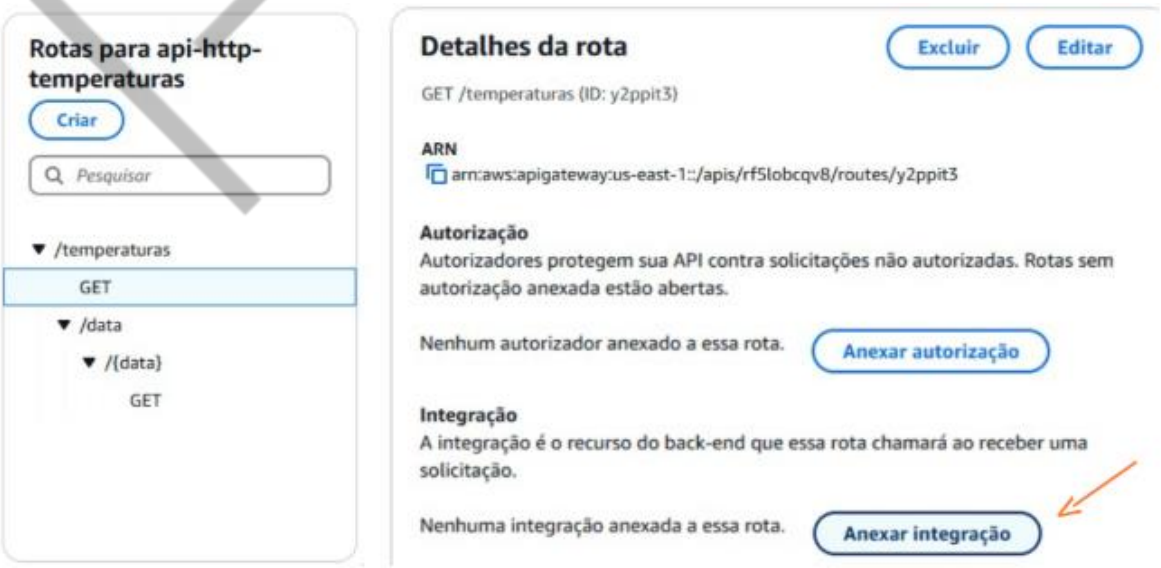

Figura 27 - Anexar Integração
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Em seguida, clique em “Criar e anexar uma integração”. Escolha no tipo de integração a opção de “Função Lambda”, e selecione a função Lambda “api-http-dynamodb” nos detalhes da integração. Depois, clique no botão “Criar”.

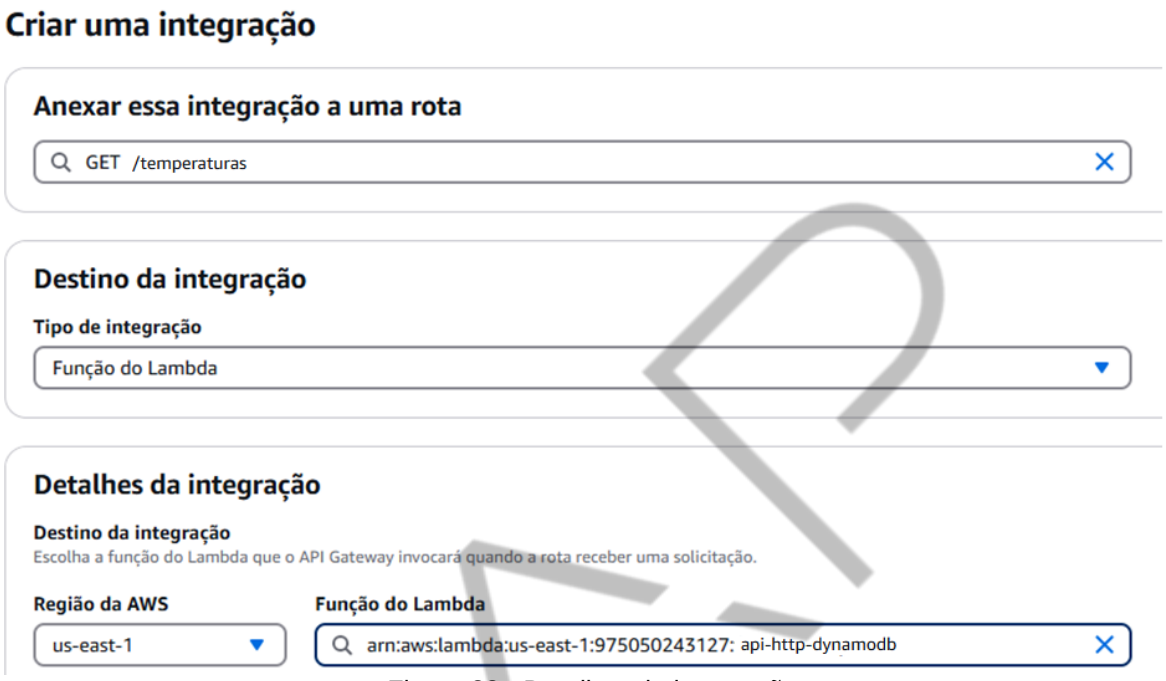

Figura 28 - Detalhes da integração
Fonte: Elaborado pelo autor (2024)

Após criar a primeira integração, selecione a outra rota da API e selecione a integração previamente criada. Finalmente clique em “Anexar Integração”.

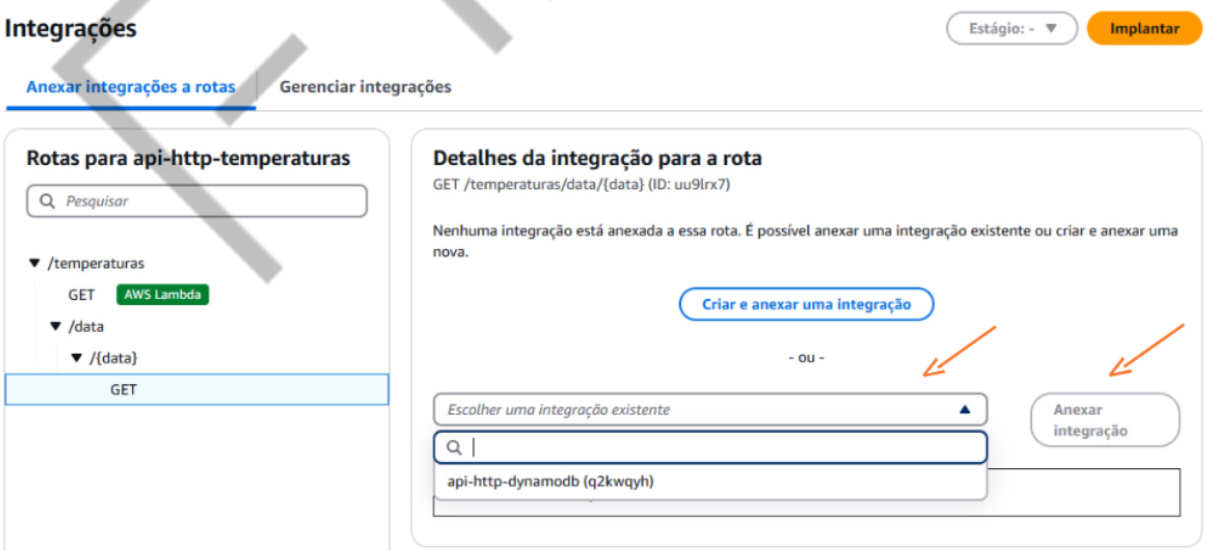

Figura 29 - Anexar integração existente
Fonte: Elaborado pelo autor (2024)

Ao terminar, você terá o seguinte cenário:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Figura 30 - Rotas com integrações com função Lambda
Fonte: Elaborado pelo autor (2024)

Agora iremos testar nossa API. Selecione o nome da API no menu lateral e copie a URL que consta abaixo do *endpoint* padrão.

Figura 31 - *Endpoint* da API
Fonte: Elaborado pelo autor (2024)

Com uma ferramenta de testes de API (exemplo: Postman) ou com o navegador, podemos simular consultas à nossa API.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

https://rf5lobcqv8.execute-api.us-east-1.amazonaws.com/temperaturas


```
[
    {
        "id_estacao": 1,
        "id": "1cd6887d-3f90-4583-9e78-df7ccbbf8ac7",
        "data": "2024-12-20 08:00:00",
        "temperatura": 22,
        "latitude": -23.5505,
        "longitude": -46.6333,
        "nome_arquivo": "estacao_1_2024_12_20.csv"
    },
    {
        "id_estacao": 1,
        "id": "c445db2e-cd64-4b3e-8283-79a473ff383f",
        "data": "2024-12-20 09:00:00",
        "temperatura": 24,
        "latitude": -23.5505,
        "longitude": -46.6333,
        "nome_arquivo": "estacao_1_2024_12_20.csv"
    },
```


Figura 32 - Simulação de consulta com o navegador
Fonte: Elaborado pelo autor (2024)

Na figura anterior, vemos a simulação da consulta geral de registros utilizando os dados da nossa API.

Podemos consultar os registros, filtrando pela data específica e acionando a outra rota criada pela API.

https://rf5lobcqv8.execute-api.us-east-1.amazonaws.com/temperaturas/data/2024-12-21


```
[
    {
        "id_estacao": 1,
        "id": "4e88c49a-d26c-4b7c-87e1-9f8035682bbe",
        "temperatura": 23,
        "latitude": -23.5505,
        "longitude": -46.6333,
        "nome_arquivo": "estacao_1_2024_12_21.csv",
        "data": "2024-12-21 08:00:00"
    },
    {
        "id_estacao": 1,
        "id": "f5d9f054-82db-4eec-b29f-eeb8c24a20d5",
        "temperatura": 25,
        "latitude": -23.5505,
        "longitude": -46.6333,
        "nome_arquivo": "estacao_1_2024_12_21.csv",
        "data": "2024-12-21 09:00:00"
    },
```


Figura 33 - Consulta de registros por data pela API
Fonte: Elaborado pelo autor (2024)

Também é possível realizar o consumo da API por meio de outros aplicativos, como o Postman, que permite acionar todos os métodos HTTP de uma API.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

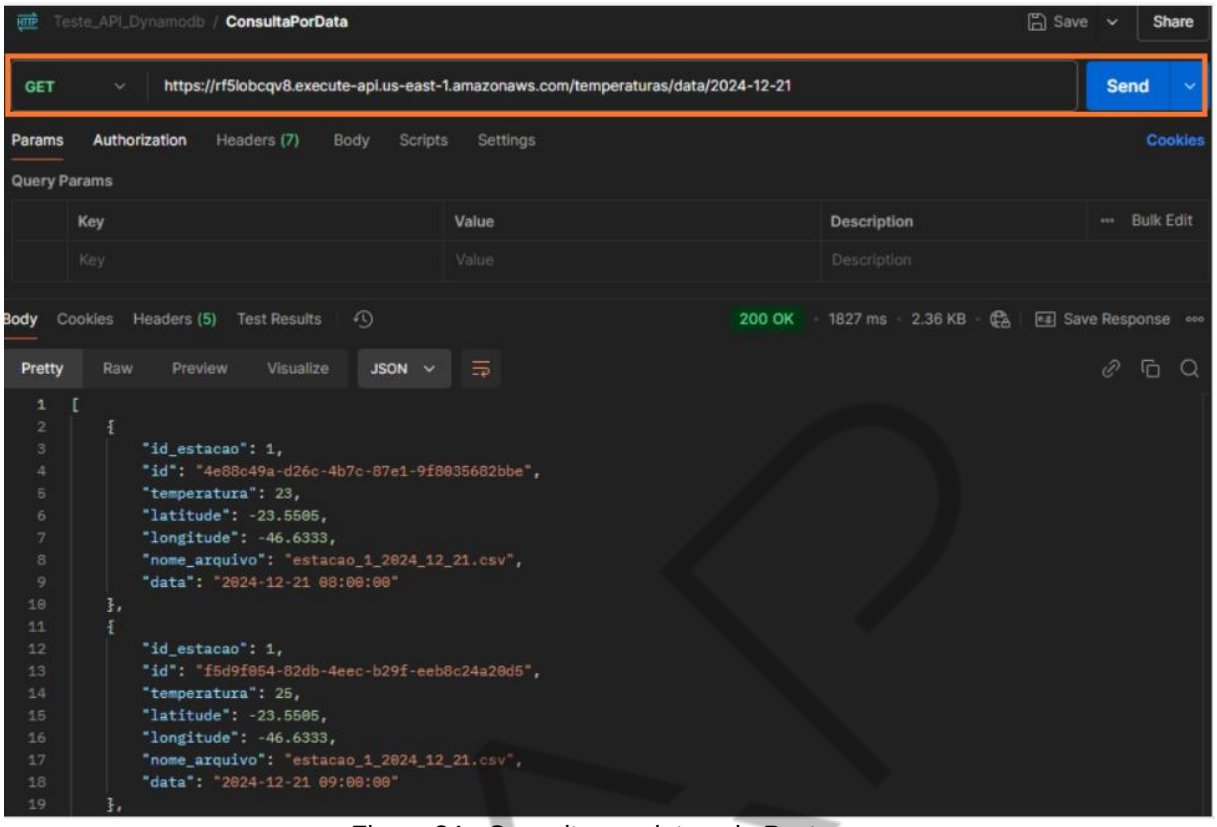

Figura 34 - Consulta por data pelo Postman
Fonte: Elaborado pelo autor (2024)

Finalizamos a nossa solução *serverless* para consumo de dados por meio de uma API. Agora, iremos aprender como excluir os recursos utilizados caso não precisemos mais desta API.

3.4 Limpeza dos recursos utilizados

Após a criação e testes da API, o ideal é excluir os recursos utilizados caso não sejam mais utilizados, como a tabela do DynamoDB e a própria API.

Para excluir a API, basta selecioná-la no menu e clicar em “Excluir”.

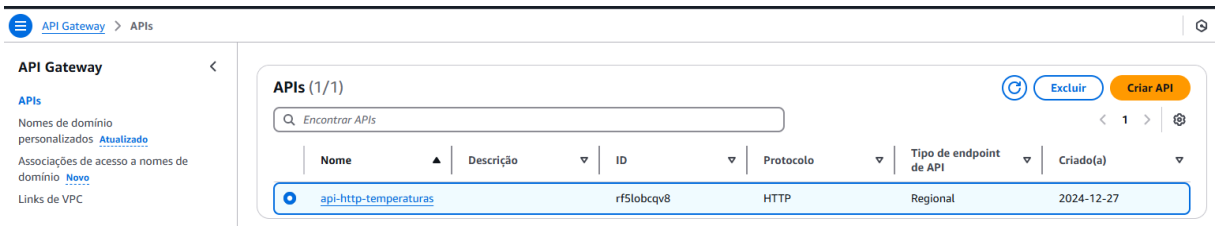

Figura 35 - Exclusão da API
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Exclua também a tabela do DynamoDB para evitar usar os recursos do ambiente de testes do Learner Lab.

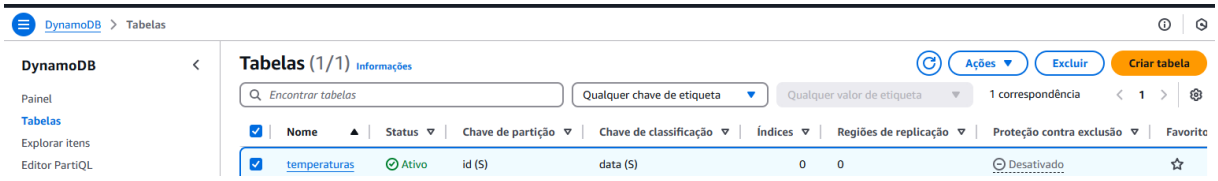

Figura 36 - Exclusão da tabela do DynamoDB
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

CONCLUSÃO

A construção de arquiteturas *serverless* utilizando API Gateway, AWS Lambda e DynamoDB oferece uma solução altamente escalável, econômica e de fácil manutenção para desenvolver APIs modernas e robustas. Essa abordagem elimina a necessidade de gerenciar a infraestrutura diretamente, permitindo que desenvolvedores foquem na lógica de negócios.

Aprendemos como implementar uma solução completa, desde a ingestão de dados, até a o consumo por meio de APIs, utilizando o API Gateway.

Em questão de minutos, construímos uma solução completa usando a AWS, permitindo fazer uma entrega rápida aos nossos clientes.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

ARQUITETURA Medalhão. **DataBricks**, 29 de agosto de 2024. Disponível em: <https://www.databricks.com/br/glossary/medallion-architecture>. Acesso em: 05 fev. 2025.

AWS Academy. **AWS Academy**, 04 de agosto de 2024. Disponível em: <https://awsacademy.instructure.com/>. Acesso em: 05 fev. 2025.

CULKIN, J.; ZAZON, M. **AWS Cookbook. Recipes for Success on AWS**. O' Reilly, 2022.

EAGAR, G. **Data Engineering with AWS**. Birmingham, UK: Packt Publishing, 2021.

ELMASRI, R.; NAVATHE, S. B. **Sistemas de Banco de Dados**. 7º Ed.; São Paulo: Pearson Education do Brasil, 2018.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

AWS	Amazon Web Services.
CLI	Command Line Interface.
EBS	Elastic Block Storage.
EC2	Elastic Compute Cloud.
EFS	Elastic File System.
RDS	Relational Database Service.
ELT	Extração, Carga e Transformação.
ELT	Extração, Transformação e Carga.
S3	Amazon Simple Storage Service.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com